

FlipTech

Rapport de développement

PROJET INFORMATIQUE 2 – PREING1

CY Tech

Développement d'un jeu de cartes stratégique en langage C

Brahim BENALI
Kais KHEMIRI
Youssef BOULMLIK

Année universitaire 2025–2026

19 mai 2026

Table des matières

1	Introduction	2
2	Architecture logicielle	2
2.1	Découpage en modules	2
2.2	Choix de gestion mémoire	3
2.3	Refactorisation principale	3
3	Organisation du travail	3
4	Détails d'implémentation	4
4.1	Module Deck	4
4.2	Module Rules	4
4.3	Interface utilisateur	4
4.4	Sauvegarde	4
5	Difficultés rencontrées	5
5.1	Cas limites	5
5.2	Saisies utilisateur	5
5.3	Fonctionnalités abandonnées	5
6	Tests et validation	6
7	Conclusion	6

1 Introduction

FlipTech est un jeu de cartes développé en langage C dans le cadre du Projet Informatique 2.

L'objectif du projet était de concevoir une application fonctionnelle, modulaire et stable, respectant des contraintes strictes : absence de variables globales, séparation entre logique métier et affichage, et fonctions de taille limitée.

Le développement a nécessité une organisation rigoureuse afin de permettre un travail en parallèle tout en garantissant une intégration cohérente du code.

Contraintes principales : fonctions limitées en taille, absence de variables globales, validation des entrées utilisateur et modularisation stricte du code.

2 Architecture logicielle

2.1 Découpage en modules

Le programme a été structuré en plusieurs modules indépendants afin de limiter les dépendances :

- `deck` : génération et gestion de la pioche
- `player` : structure et état des joueurs
- `rules` : logique du jeu et traitement des cartes
- `game` : boucle principale et gestion des manches
- `ui` : affichage et interactions utilisateur
- `save/stats` : sauvegarde et statistiques
- `utils` : fonctions auxiliaires

Chaque module expose uniquement les fonctions nécessaires via des fichiers d'en-tête, garantissant une séparation claire des responsabilités.

2.2 Choix de gestion mémoire

Le projet utilise exclusivement des structures à taille fixe :

- Deck : 85 cartes maximum
- Main joueur : 20 cartes

L'allocation dynamique a été volontairement écartée afin de réduire les risques d'erreurs (fuites mémoire, pointeurs invalides, segmentation faults) et simplifier la gestion globale du programme.

Le choix des structures statiques a été privilégié pour garantir la stabilité et la prévisibilité du comportement du programme.

2.3 Refactorisation principale

Dans les premières versions, la logique de jeu était dispersée dans la boucle principale. Cela rendait le code difficile à maintenir.

Une refactorisation a conduit à centraliser le traitement dans la fonction `rules_apply_card`, devenue le point d'entrée unique pour la logique métier.

3 Organisation du travail

Le projet a été réparti entre les membres afin de favoriser le développement parallèle :

- Brahim : module Deck et génération aléatoire
- Kais : moteur de jeu (Rules, Game, Player)
- Youssef : interface utilisateur et système de sauvegarde

Git a été utilisé comme outil de centralisation du code, sans workflow complexe. La coordination s'est faite de manière régulière afin d'assurer la compatibilité des modules.

4 Détails d'implémentation

4.1 Module Deck

Le deck est généré de manière déterministe selon les règles définies, puis mélangé via l'algorithme de Fisher-Yates.

La fonction de pioche retourne une valeur spéciale lorsque le deck est vide afin d'éviter les erreurs de contrôle dans la boucle principale.

4.2 Module Rules

Le module Rules constitue le cœur du programme. Il gère :

- l'ajout de cartes dans la main
- la détection des doublons
- les conditions de victoire
- les effets de bonus

Il retourne un état de jeu standardisé permettant au module Game de gérer le déroulement des manches.

4.3 Interface utilisateur

Les entrées utilisateur ont initialement été implémentées avec `scanf`, mais des problèmes de gestion de buffer ont conduit à une migration vers `fgets` avec validation manuelle.

L'affichage utilise des codes ANSI pour distinguer les effets visuels liés aux cartes bonus.

4.4 Sauvegarde

Les données sont stockées dans un fichier texte au format simple :

```
nom | score total | score manche
```

Ce format a été retenu pour sa simplicité et sa facilité d'exploitation.

5 Difficultés rencontrées

5.1 Cas limites

Plusieurs cas limites ont nécessité des ajustements :

- gestion des joueurs quittant simultanément une manche
- traitement de la pioche vide
- correction des doublons détectés trop tardivement

5.2 Saisies utilisateur

La gestion des entrées a été un point critique du projet. Les erreurs de saisie provoquaient des comportements instables avec `scanf`.

Le passage à une validation manuelle via `fgets` a permis de stabiliser le système.

Une grande partie des instabilités initiales provenait de la gestion incorrecte des entrées utilisateur.

5.3 Fonctionnalités abandonnées

Certaines fonctionnalités ont été abandonnées afin de garantir la stabilité :

- intelligence artificielle simple
- cartes spéciales supplémentaires
- sauvegarde complète de partie

6 Tests et validation

Cas testé	Résultat
Doublon	Perte immédiate de manche
7 cartes différentes	Victoire validée (+ 15 points)
Pioche vide	Fin correcte de partie
Bonus cumulés	Calcul correct
Entrées invalides	Requête répétée sans crash

Les tests ont été réalisés sur plusieurs configurations (2 à 4 joueurs) sans anomalie critique.

7 Conclusion

Le projet FlipTech a permis de mettre en œuvre une architecture modulaire en langage C et de comprendre les enjeux liés à la stabilité logicielle.

Les principales difficultés rencontrées concernent la gestion des cas limites, l'intégration des modules et la robustesse des entrées utilisateur.

Le programme final répond aux exigences initiales en termes de structure et de fiabilité.

Brahim BENALI
Kais KHEMIRI
Youssef BOULMLIK
19 mai 2026